

# Project 05: CRM System (Customer Relationship Management)

Difficulty: Intermediate | Estimated Time: 2 Weeks

---

## 1. Project Overview and Goals

This project builds a CRM database backend similar to Salesforce or HubSpot. It covers contact and company management, deal pipeline tracking, activity logging, task management, email campaigns, and sales performance reporting. The CRM is the backbone of any sales-driven organization, and its database is rich with temporal data, hierarchical relationships, and funnel analytics.

### Goals:

- Model a B2B/B2C contact-company hierarchy with flexible custom fields.
  - Implement a deal pipeline with stage-based probability scoring.
  - Track all interaction history (calls, emails, meetings, notes) per contact.
  - Build a sales funnel report showing conversion at each pipeline stage.
  - Use JSONB for custom field storage and activity metadata.
- 

## 2. Learning Objectives

- Design a contact-company-deal hierarchy with self-referencing contacts.
  - Use JSONB for flexible custom field storage.
  - Implement pipeline stage ordering with sort keys and win probability.
  - Write funnel analysis queries using window functions and CTEs.
  - Practice SCD (Slowly Changing Dimension) patterns for deal stage history.
  - Build a task management module with due-date and assignment queries.
  - Use `INTERVAL` arithmetic for time-to-close and SLA calculations.
  - Write email campaign performance queries with open/click rates.
- 

## 3. Functional Requirements

- **Contacts:** First name, last name, email, phone, job title, linked company.
  - **Companies:** Organization profiles with industry, size, and annual revenue.
  - **Deals:** Opportunity tracking with pipeline stage, amount, owner, close date.
  - **Activities:** Log calls, emails, meetings, notes linked to contacts/companies/deals.
  - **Tasks:** To-do items with due dates, priority, and completion tracking.
  - **Pipeline Stages:** Configurable stages with order and win probability.
  - **Email Campaigns:** Campaign metadata and contact-level engagement tracking.
  - **Reports:** Win rate, average deal size, time-to-close, activity by rep.
- 

## 4. Non-Functional Requirements

- Contacts and Companies use soft deletes ( `deleted_at` ).
- Deal amounts stored as `NUMERIC(14,2)` .
- All stage transitions logged in a `deal_stage_history` table.
- Activities are immutable once created (append-only log).

- Email bounce and unsubscribe flags stored per contact.

---

## 5. Complete Database Schema

```
-- =====
-- CRM SYSTEM - Complete Schema
-- PostgreSQL 15+
-- =====

CREATE SCHEMA IF NOT EXISTS crm;
SET search_path = crm, public;

-- -----
-- USERS (sales reps, managers)
-- -----

CREATE TABLE users (
    user_id      SERIAL PRIMARY KEY,
    email        VARCHAR(300) NOT NULL UNIQUE,
    first_name   VARCHAR(100) NOT NULL,
    last_name    VARCHAR(100) NOT NULL,
    role         VARCHAR(50) NOT NULL DEFAULT 'Sales Rep'
                CHECK (role IN ('Sales Rep', 'Senior Rep', 'Manager', 'Admin')),
    manager_id   INTEGER REFERENCES users(user_id),
    is_active    BOOLEAN NOT NULL DEFAULT TRUE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO crm.users (email, first_name, last_name, role) VALUES
('john.smith@crm.io', 'John', 'Smith', 'Manager'),
('sara.jones@crm.io', 'Sara', 'Jones', 'Sales Rep'),
('mike.wang@crm.io', 'Mike', 'Wang', 'Sales Rep'),
('lisa.chen@crm.io', 'Lisa', 'Chen', 'Senior Rep');

UPDATE crm.users SET manager_id = 1 WHERE user_id IN (2,3,4);

-- -----
-- COMPANIES
-- -----

CREATE TABLE companies (
    company_id   SERIAL PRIMARY KEY,
    name         VARCHAR(300) NOT NULL,
    website      VARCHAR(300),
    industry     VARCHAR(100),
    size_category VARCHAR(20)
                CHECK (size_category IN ('SMB', 'Mid-Market', 'Enterprise')),
    employee_count INTEGER,
    annual_revenue NUMERIC(14,2),
    country      VARCHAR(100),
    city         VARCHAR(100),
    owner_id     INTEGER REFERENCES users(user_id),
```

```

        custom_fields JSONB NOT NULL DEFAULT '{}',
        created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        deleted_at    TIMESTAMPTZ
    );

CREATE INDEX idx_companies_owner    ON companies (owner_id);
CREATE INDEX idx_companies_active  ON companies (deleted_at) WHERE deleted_at IS
NULL;
CREATE INDEX idx_companies_custom  ON companies USING GIN (custom_fields);

-- -----
-- CONTACTS
-- -----

CREATE TABLE contacts (
    contact_id    SERIAL PRIMARY KEY,
    first_name    VARCHAR(100) NOT NULL,
    last_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(300) UNIQUE,
    phone         VARCHAR(30),
    mobile        VARCHAR(30),
    job_title     VARCHAR(200),
    company_id    INTEGER REFERENCES companies(company_id),
    owner_id      INTEGER REFERENCES users(user_id),
    manager_id    INTEGER REFERENCES contacts(contact_id), -- reports to
    lead_source   VARCHAR(50) CHECK (lead_source IN (
        'Website', 'Referral', 'Cold Call', 'LinkedIn', 'Event',
        'Inbound', 'Paid Ad', 'Partner', 'Other')),
    lifecycle_stage VARCHAR(30) NOT NULL DEFAULT 'Lead'
        CHECK (lifecycle_stage IN
('Lead', 'MQL', 'SQL', 'Opportunity', 'Customer', 'Churned')),
    is_subscribed BOOLEAN NOT NULL DEFAULT TRUE,
    email_bounced BOOLEAN NOT NULL DEFAULT FALSE,
    custom_fields JSONB NOT NULL DEFAULT '{}',
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at    TIMESTAMPTZ
);

CREATE INDEX idx_contacts_company ON contacts (company_id);
CREATE INDEX idx_contacts_owner  ON contacts (owner_id);
CREATE INDEX idx_contacts_email  ON contacts (email);
CREATE INDEX idx_contacts_stage  ON contacts (lifecycle_stage);
CREATE INDEX idx_contacts_fts    ON contacts USING GIN (
    to_tsvector('english', first_name || ' ' || last_name || ' ' ||
COALESCE(job_title, ''))
);

-- -----
-- PIPELINE STAGES
-- -----

CREATE TABLE pipeline_stages (
    stage_id      SERIAL PRIMARY KEY,
    name          VARCHAR(100) NOT NULL UNIQUE,

```

```

        sort_order          SMALLINT NOT NULL UNIQUE,
        win_probability      SMALLINT NOT NULL DEFAULT 0 CHECK (win_probability BETWEEN 0 AND
100),
        is_closed_won       BOOLEAN NOT NULL DEFAULT FALSE,
        is_closed_lost      BOOLEAN NOT NULL DEFAULT FALSE
    );

```

```

INSERT INTO pipeline_stages (name, sort_order, win_probability, is_closed_won,
is_closed_lost)

```

```
VALUES

```

```

    ('Prospecting',        1,  10, FALSE, FALSE),
    ('Qualification',      2,  20, FALSE, FALSE),
    ('Needs Analysis',     3,  30, FALSE, FALSE),
    ('Value Proposition',  4,  50, FALSE, FALSE),
    ('Decision Makers',    5,  60, FALSE, FALSE),
    ('Proposal/Price Quote',6,  75, FALSE, FALSE),
    ('Negotiation',        7,  90, FALSE, FALSE),
    ('Closed Won',         8, 100, TRUE,  FALSE),
    ('Closed Lost',        9,   0, FALSE, TRUE);

```

```

-- -----
-- DEALS
-- -----

```

```
CREATE TABLE deals (

```

```

    deal_id          SERIAL PRIMARY KEY,
    name             VARCHAR(300) NOT NULL,
    company_id       INTEGER REFERENCES companies(company_id),
    contact_id       INTEGER REFERENCES contacts(contact_id),
    owner_id         INTEGER NOT NULL REFERENCES users(user_id),
    stage_id         INTEGER NOT NULL REFERENCES pipeline_stages(stage_id),
    amount           NUMERIC(14,2) NOT NULL DEFAULT 0 CHECK (amount >= 0),
    currency         VARCHAR(3) NOT NULL DEFAULT 'USD',
    close_date       DATE NOT NULL,
    probability       SMALLINT,
    description       TEXT,
    lost_reason       VARCHAR(200),
    source           VARCHAR(50),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    closed_at        TIMESTAMPTZ

```

```
);

```

```

CREATE INDEX idx_deals_company ON deals (company_id);
CREATE INDEX idx_deals_contact ON deals (contact_id);
CREATE INDEX idx_deals_owner   ON deals (owner_id);
CREATE INDEX idx_deals_stage   ON deals (stage_id);
CREATE INDEX idx_deals_close   ON deals (close_date);

```

```

-- -----
-- DEAL STAGE HISTORY (SCD audit)
-- -----

```

```
CREATE TABLE deal_stage_history (

```

```

    history_id BIGSERIAL PRIMARY KEY,
    deal_id    INTEGER NOT NULL REFERENCES deals(deal_id),
    from_stage_id INTEGER REFERENCES pipeline_stages(stage_id),
    to_stage_id INTEGER NOT NULL REFERENCES pipeline_stages(stage_id),
    changed_by INTEGER REFERENCES users(user_id),
    changed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    time_in_stage INTERVAL -- set when leaving stage
);

CREATE INDEX idx_stage_history_deal ON deal_stage_history (deal_id, changed_at);

-----
-- ACTIVITY TYPES
-----

CREATE TABLE activity_types (
    type_id SERIAL PRIMARY KEY,
    code    VARCHAR(20) NOT NULL UNIQUE,
    name    VARCHAR(50) NOT NULL,
    icon    VARCHAR(50)
);

INSERT INTO activity_types (code, name) VALUES
('CALL',    'Phone Call'),
('EMAIL',   'Email'),
('MEETING', 'Meeting'),
('NOTE',    'Note'),
('DEMO',    'Demo'),
('TASK',    'Task Completed');

-----
-- ACTIVITIES (immutable log)
-----

CREATE TABLE activities (
    activity_id BIGSERIAL PRIMARY KEY,
    type_id     INTEGER NOT NULL REFERENCES activity_types(type_id),
    user_id     INTEGER NOT NULL REFERENCES users(user_id),
    contact_id  INTEGER REFERENCES contacts(contact_id),
    company_id  INTEGER REFERENCES companies(company_id),
    deal_id     INTEGER REFERENCES deals(deal_id),
    subject     VARCHAR(400),
    description  TEXT,
    duration_min SMALLINT CHECK (duration_min >= 0),
    metadata    JSONB NOT NULL DEFAULT '{}',
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_activity_linked CHECK (
        contact_id IS NOT NULL OR company_id IS NOT NULL OR deal_id IS NOT NULL
    )
);

CREATE INDEX idx_activities_contact ON activities (contact_id, occurred_at DESC);
CREATE INDEX idx_activities_deal    ON activities (deal_id, occurred_at DESC);
CREATE INDEX idx_activities_user    ON activities (user_id, occurred_at DESC);

```

```

CREATE INDEX idx_activities_type    ON activities (type_id);

-- -----
-- TASKS
-- -----

CREATE TABLE tasks (
    task_id        SERIAL PRIMARY KEY,
    title          VARCHAR(400) NOT NULL,
    description     TEXT,
    assigned_to    INTEGER NOT NULL REFERENCES users(user_id),
    created_by     INTEGER NOT NULL REFERENCES users(user_id),
    contact_id     INTEGER REFERENCES contacts(contact_id),
    deal_id        INTEGER REFERENCES deals(deal_id),
    priority       VARCHAR(10) NOT NULL DEFAULT 'Medium'
                  CHECK (priority IN ('Low', 'Medium', 'High', 'Critical')),
    status         VARCHAR(20) NOT NULL DEFAULT 'Open'
                  CHECK (status IN ('Open', 'In Progress', 'Completed', 'Cancelled')),
    due_date       TIMESTAMPTZ,
    completed_at   TIMESTAMPTZ,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_tasks_assigned ON tasks (assigned_to, due_date);
CREATE INDEX idx_tasks_deal    ON tasks (deal_id);
CREATE INDEX idx_tasks_open    ON tasks (assigned_to) WHERE status = 'Open';

-- -----
-- EMAIL CAMPAIGNS
-- -----

CREATE TABLE campaigns (
    campaign_id    SERIAL PRIMARY KEY,
    name           VARCHAR(300) NOT NULL,
    subject        VARCHAR(500),
    status         VARCHAR(20) NOT NULL DEFAULT 'Draft'
                  CHECK (status IN
('Draft', 'Scheduled', 'Sending', 'Sent', 'Cancelled')),
    sent_at        TIMESTAMPTZ,
    created_by     INTEGER NOT NULL REFERENCES users(user_id),
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE campaign_contacts (
    id             BIGSERIAL PRIMARY KEY,
    campaign_id    INTEGER NOT NULL REFERENCES campaigns(campaign_id),
    contact_id     INTEGER NOT NULL REFERENCES contacts(contact_id),
    sent_at        TIMESTAMPTZ,
    opened_at      TIMESTAMPTZ,
    clicked_at     TIMESTAMPTZ,
    bounced_at     TIMESTAMPTZ,
    unsubscribed_at TIMESTAMPTZ,
    UNIQUE (campaign_id, contact_id)
);

```

```
CREATE INDEX idx_campaign_contacts ON campaign_contacts (campaign_id);
```

## 6. ASCII ER Diagram

```

USERS (self-ref for manager_id)
+-----+
| user_id (PK)      | <--+
| email (UNIQUE)    |   | manager_id
| role              | ----+
+-----+-----+

          |owner_id          PIPELINE_STAGES
          |                  +-----+
          +-----+          | stage_id (PK) |
          |          |          | name (UNIQUE) |
          v          v          | sort_order  |
COMPANIES  CONTACTS          | win_probability |
+-----+ +-----+ +-----+-----+
| company_id| | contact_id|          |
| name      | | first_name|          |
| industry  | | email      |          v
| owner_id  | | company_id|          DEALS
| custom_   | | owner_id  | +-----+-----+
| fields    | | lifecycle | | deal_id (PK)  |
| (JSONB)   | | _stage   | | name          |
+-----+ +-----+ | company_id (FK) |
          |          | | contact_id (FK) |
          |          | | owner_id (FK)  |
          +-----+ | stage_id (FK)  |
          |          | | amount        |
          v          +-----+-----+
          ACTIVITIES          |
+-----+-----+          |
| activity_id (PK) |          DEAL_STAGE_HISTORY
| type_id (FK)    | +-----+-----+
| user_id (FK)    | | history_id (PK) |
| contact_id (FK) | | deal_id (FK)  |
| company_id (FK) | | from_stage_id |
| deal_id (FK)    | | to_stage_id  |
| occurred_at     | | changed_at   |
+-----+-----+ +-----+-----+

TASKS          CAMPAIGNS
+-----+-----+ +-----+-----+
| task_id (PK)   | | campaign_id (PK) |
| assigned_to(FK)| | name            |
| deal_id (FK)   | +-----+-----+
| priority       | |          |
| due_date       |          v
+-----+-----+ CAMPAIGN_CONTACTS

```

```
+-----+
| campaign_id (FK) |
| contact_id (FK)  |
| opened_at        |
| clicked_at       |
+-----+
```

## 7. Sample Data Scripts

```
SET search_path = crm, public;

-- Companies
INSERT INTO companies (name, industry, size_category, annual_revenue, country,
owner_id) VALUES
('Acme Technologies', 'Technology', 'Enterprise', 50000000, 'USA', 2),
('BlueSky Analytics', 'Analytics', 'Mid-Market', 8000000, 'UK', 3),
('GreenLeaf Retail', 'Retail', 'SMB', 1500000, 'USA', 4),
('NovaCorp Industries', 'Manufacturing', 'Enterprise', 200000000, 'Germany', 2);

-- Contacts
INSERT INTO contacts (first_name, last_name, email, job_title, company_id, owner_id,
lead_source, lifecycle_stage) VALUES
('James', 'Carter', 'j.carter@acme.com', 'CTO', 1, 2,
'LinkedIn', 'SQL'),
('Rachel', 'Kim', 'r.kim@acme.com', 'VP Engineering', 1, 2,
'Referral', 'Opportunity'),
('Tom', 'Nakamura', 't.nakamura@bluesky.com', 'CEO', 2, 3, 'Cold
Call', 'MQL'),
('Priya', 'Sharma', 'p.sharma@greenleaf.com', 'Operations Lead', 3, 4,
'Website', 'Lead'),
('Karl', 'Mueller', 'k.mueller@novacorp.de', 'IT Director', 4, 2, 'Event',
'SQL');

-- Deals
INSERT INTO deals (name, company_id, contact_id, owner_id, stage_id, amount,
close_date) VALUES
('Acme - Enterprise License', 1, 2, 2, 6, 120000.00, '2024-12-31'),
('BlueSky - Analytics Platform', 2, 3, 3, 4, 45000.00, '2024-11-30'),
('GreenLeaf - POS Upgrade', 3, 4, 4, 2, 12000.00, '2024-10-15'),
('NovaCorp - Full Suite', 4, 5, 2, 7, 380000.00, '2024-12-15'),
('Acme - Support Contract', 1, 1, 2, 8, 24000.00, '2024-09-01');

-- Activities
INSERT INTO activities (type_id, user_id, contact_id, deal_id, subject,
duration_min, occurred_at) VALUES
(1, 2, 2, 1, 'Discovery call - Acme CTO', 45, NOW() - INTERVAL '5
days'),
(3, 2, 2, 1, 'Product demo - Acme team', 90, NOW() - INTERVAL '3
days'),
(2, 3, 3, 2, 'Follow-up email sent', 0, NOW() - INTERVAL '7
```



```

days'),
  (4, 4, 4, 3, 'Note: Customer interested in mobile', NULL, NOW() - INTERVAL '2
days'),
  (1, 2, 5, 4, 'Negotiation call with NovaCorp', 60, NOW() - INTERVAL '1 day');

-- Tasks
INSERT INTO tasks (title, assigned_to, created_by, contact_id, deal_id, priority,
due_date) VALUES
  ('Send proposal to Acme', 2, 1, 2, 1, 'High', NOW() + INTERVAL '2
days'),
  ('Follow up with BlueSky CEO', 3, 1, 3, 2, 'Medium', NOW() + INTERVAL '5
days'),
  ('Schedule GreenLeaf site visit', 4, 1, 4, 3, 'Low', NOW() + INTERVAL '10
days'),
  ('Finalize NovaCorp contract', 2, 1, 5, 4, 'Critical', NOW() + INTERVAL '3
days');

-- Deal stage history
INSERT INTO deal_stage_history (deal_id, from_stage_id, to_stage_id, changed_by,
changed_at) VALUES
  (1, NULL, 1, 2, NOW() - INTERVAL '30 days'),
  (1, 1, 3, 2, NOW() - INTERVAL '20 days'),
  (1, 3, 6, 2, NOW() - INTERVAL '5 days');

-- Campaign
INSERT INTO campaigns (name, subject, status, created_by, sent_at) VALUES
  ('Q4 Product Launch', 'Introducing Our New Platform - Built for Enterprise',
'Sent', 1, NOW() - INTERVAL '3 days');

INSERT INTO campaign_contacts (campaign_id, contact_id, sent_at, opened_at,
clicked_at) VALUES
  (1, 1, NOW()-INTERVAL '3 days', NOW()-INTERVAL '2 days', NOW()-INTERVAL '1 day'),
  (1, 2, NOW()-INTERVAL '3 days', NOW()-INTERVAL '2 days', NULL),
  (1, 3, NOW()-INTERVAL '3 days', NULL, NULL),
  (1, 4, NOW()-INTERVAL '3 days', NOW()-INTERVAL '3 days', NOW()-INTERVAL '3 days');

```

## 8. Core SQL Queries

```

SET search_path = crm, public;

-- -----
-- Q1: Sales pipeline overview (deals by stage)
-- -----

SELECT
  ps.name                                AS stage,
  ps.win_probability                     AS probability_pct,
  COUNT(d.deal_id)                       AS deal_count,
  ROUND(SUM(d.amount), 2)                 AS total_amount,
  ROUND(SUM(d.amount * ps.win_probability / 100.0), 2) AS weighted_amount
FROM pipeline_stages ps

```

```

LEFT JOIN deals d ON ps.stage_id = d.stage_id
WHERE NOT ps.is_closed_won AND NOT ps.is_closed_lost
GROUP BY ps.stage_id, ps.name, ps.win_probability, ps.sort_order
ORDER BY ps.sort_order;

```

```

-- -----
-- Q2: Rep performance dashboard
-- -----

```

```

SELECT
    u.first_name || ' ' || u.last_name AS rep,
    COUNT(d.deal_id) AS total_deals,
    COUNT(d.deal_id) FILTER (WHERE ps.is_closed_won) AS won,
    COUNT(d.deal_id) FILTER (WHERE ps.is_closed_lost) AS lost,
    ROUND(SUM(d.amount) FILTER (WHERE ps.is_closed_won), 2) AS revenue_won,
    ROUND(AVG(d.amount) FILTER (WHERE ps.is_closed_won), 2) AS avg_deal_size,
    ROUND(
        100.0 * COUNT(d.deal_id) FILTER (WHERE ps.is_closed_won)
        / NULLIF(COUNT(d.deal_id) FILTER (WHERE ps.is_closed_won OR
ps.is_closed_lost), 0)
        , 1) AS win_rate_pct
FROM users u
LEFT JOIN deals d ON u.user_id = d.owner_id
LEFT JOIN pipeline_stages ps ON d.stage_id = ps.stage_id
GROUP BY u.user_id, u.first_name, u.last_name
ORDER BY revenue_won DESC NULLS LAST;

```

```

-- -----
-- Q3: Deals closing this month - pipeline report
-- -----

```

```

SELECT
    d.name AS deal,
    c.name AS company,
    u.first_name || ' ' || u.last_name AS owner,
    ps.name AS stage,
    d.amount,
    d.close_date,
    d.close_date - CURRENT_DATE AS days_to_close,
    ps.win_probability
FROM deals d
JOIN companies c ON d.company_id = c.company_id
JOIN users u ON d.owner_id = u.user_id
JOIN pipeline_stages ps ON d.stage_id = ps.stage_id
WHERE d.close_date BETWEEN CURRENT_DATE AND DATE_TRUNC('month', CURRENT_DATE) +
INTERVAL '1 month - 1 day'
AND NOT ps.is_closed_won AND NOT ps.is_closed_lost
ORDER BY d.amount DESC;

```

```

-- -----
-- Q4: Average time spent per pipeline stage
-- -----

```

```

SELECT
    ps.name AS stage,

```

```

        COUNT(dsh.history_id)                AS transitions,
        AVG(dsh.time_in_stage)                AS avg_time_in_stage
FROM deal_stage_history dsh
JOIN pipeline_stages ps ON dsh.from_stage_id = ps.stage_id
WHERE dsh.time_in_stage IS NOT NULL
GROUP BY ps.stage_id, ps.name, ps.sort_order
ORDER BY ps.sort_order;

```

```

-- -----
-- Q5: Contact timeline (all activities for a contact)
-- -----

```

```

SELECT
    a.occurred_at::DATE                AS date,
    at2.name                           AS activity_type,
    a.subject,
    u.first_name || ' ' || u.last_name AS rep,
    a.duration_min,
    COALESCE(d.name, '(No Deal)') AS deal
FROM activities a
JOIN activity_types at2 ON a.type_id = at2.type_id
JOIN users u          ON a.user_id = u.user_id
LEFT JOIN deals d      ON a.deal_id = d.deal_id
WHERE a.contact_id = 2
ORDER BY a.occurred_at DESC;

```

```

-- -----
-- Q6: Email campaign performance
-- -----

```

```

SELECT
    camp.name                AS campaign,
    COUNT(cc.id)              AS sent,
    COUNT(cc.opened_at)       AS opened,
    COUNT(cc.clicked_at)      AS clicked,
    COUNT(cc.bounced_at)     AS bounced,
    COUNT(cc.unsubscribed_at) AS unsubscribed,
    ROUND(100.0 * COUNT(cc.opened_at) / NULLIF(COUNT(cc.id), 0), 1) AS
open_rate_pct,
    ROUND(100.0 * COUNT(cc.clicked_at) / NULLIF(COUNT(cc.opened_at), 0), 1) AS
ctr_pct
FROM campaigns camp
LEFT JOIN campaign_contacts cc ON camp.campaign_id = cc.campaign_id
WHERE camp.status = 'Sent'
GROUP BY camp.campaign_id, camp.name
ORDER BY camp.sent_at DESC;

```

```

-- -----
-- Q7: Tasks overdue by rep
-- -----

```

```

SELECT
    u.first_name || ' ' || u.last_name AS rep,
    COUNT(*) FILTER (WHERE t.due_date < NOW() AND t.status = 'Open') AS
overdue_tasks,

```

```

COUNT(*) FILTER (WHERE t.status = 'Open') AS open_tasks,
COUNT(*) FILTER (WHERE t.status = 'Completed'
AND t.completed_at >= NOW() - INTERVAL '7 days') AS
completed_this_week
FROM users u
LEFT JOIN tasks t ON u.user_id = t.assigned_to
GROUP BY u.user_id, u.first_name, u.last_name
ORDER BY overdue_tasks DESC;

-- -----
-- Q8: Deal velocity (time from created to closed won)
-- -----
SELECT
    d.name AS deal,
    d.amount,
    u.first_name || ' ' || u.last_name AS rep,
    d.created_at::DATE AS opened,
    d.closed_at::DATE AS closed,
    (d.closed_at::DATE - d.created_at::DATE) AS days_to_close
FROM deals d
JOIN users u ON d.owner_id = u.user_id
JOIN pipeline_stages ps ON d.stage_id = ps.stage_id
WHERE ps.is_closed_won AND d.closed_at IS NOT NULL
ORDER BY days_to_close;

-- -----
-- Q9: Company engagement score (activity count)
-- -----
SELECT
    c.name AS company,
    c.size_category,
    COUNT(a.activity_id) AS total_activities,
    COUNT(a.activity_id) FILTER (WHERE at2.code = 'CALL') AS calls,
    COUNT(a.activity_id) FILTER (WHERE at2.code = 'MEETING') AS meetings,
    MAX(a.occurred_at)::DATE AS last_activity,
    CURRENT_DATE - MAX(a.occurred_at)::DATE AS days_since_contact
FROM companies c
LEFT JOIN activities a ON c.company_id = a.company_id
LEFT JOIN activity_types at2 ON a.type_id = at2.type_id
WHERE c.deleted_at IS NULL
GROUP BY c.company_id, c.name, c.size_category
ORDER BY total_activities DESC;

-- -----
-- Q10: Funnel conversion rates (lifecycle stages)
-- -----
SELECT
    lifecycle_stage,
    COUNT(*) AS contacts,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1) AS pct_of_total,
    ROUND(100.0 * COUNT(*) / FIRST_VALUE(COUNT(*)) OVER (ORDER BY
CASE lifecycle_stage

```

```

        WHEN 'Lead' THEN 1
        WHEN 'MQL' THEN 2
        WHEN 'SQL' THEN 3
        WHEN 'Opportunity' THEN 4
        WHEN 'Customer' THEN 5
        WHEN 'Churned' THEN 6
    END
), 1) AS pct_of_leads
FROM contacts
WHERE deleted_at IS NULL
GROUP BY lifecycle_stage
ORDER BY CASE lifecycle_stage
    WHEN 'Lead' THEN 1 WHEN 'MQL' THEN 2 WHEN 'SQL' THEN 3
    WHEN 'Opportunity' THEN 4 WHEN 'Customer' THEN 5 WHEN 'Churned' THEN 6
END;

-- -----
-- Q11: Won vs Lost analysis by source
-- -----

SELECT
    d.source,
    COUNT(*) FILTER (WHERE ps.is_closed_won) AS won,
    COUNT(*) FILTER (WHERE ps.is_closed_lost) AS lost,
    ROUND(SUM(d.amount) FILTER (WHERE ps.is_closed_won), 2) AS won_revenue,
    ROUND(100.0 * COUNT(*) FILTER (WHERE ps.is_closed_won)
        / NULLIF(COUNT(*) FILTER (WHERE ps.is_closed_won OR ps.is_closed_lost),
0), 1) AS win_rate_pct
FROM deals d
JOIN pipeline_stages ps ON d.stage_id = ps.stage_id
WHERE ps.is_closed_won OR ps.is_closed_lost
GROUP BY d.source
ORDER BY won_revenue DESC NULLS LAST;

-- -----
-- Q12: Next best actions for open deals (tasks due)
-- -----

SELECT
    d.name AS deal,
    c.name AS company,
    ps.name AS current_stage,
    d.amount,
    t.title AS next_task,
    t.priority,
    t.due_date,
    CASE WHEN t.due_date < NOW() THEN 'OVERDUE'
        WHEN t.due_date < NOW() + INTERVAL '3 days' THEN 'DUE SOON'
        ELSE 'On Track'
    END AS task_status
END AS task_status
FROM deals d
JOIN companies c ON d.company_id = c.company_id
JOIN pipeline_stages ps ON d.stage_id = ps.stage_id
LEFT JOIN tasks t ON d.deal_id = t.deal_id AND t.status = 'Open'

```

```
WHERE NOT ps.is_closed_won AND NOT ps.is_closed_lost
ORDER BY t.due_date NULLS LAST;
```

## 9. Stored Procedures / Functions

```
-- -----
-- FUNCTION 1: Advance deal to next stage
-- -----

CREATE OR REPLACE FUNCTION crm.advance_deal_stage(
    p_deal_id    INTEGER,
    p_user_id    INTEGER,
    p_new_stage  INTEGER DEFAULT NULL    -- NULL = auto-advance to next
)
RETURNS TEXT
LANGUAGE plpgsql AS $$
DECLARE
    v_deal        crm.deals%ROWTYPE;
    v_old_stage    crm.pipeline_stages%ROWTYPE;
    v_new_stage    crm.pipeline_stages%ROWTYPE;
    v_time_spent  INTERVAL;
BEGIN
    SELECT * INTO v_deal FROM crm.deals WHERE deal_id = p_deal_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Deal % not found', p_deal_id; END IF;

    SELECT * INTO v_old_stage FROM crm.pipeline_stages WHERE stage_id =
v_deal.stage_id;

    IF v_old_stage.is_closed_won OR v_old_stage.is_closed_lost THEN
        RAISE EXCEPTION 'Deal is already closed';
    END IF;

    IF p_new_stage IS NOT NULL THEN
        SELECT * INTO v_new_stage FROM crm.pipeline_stages WHERE stage_id =
p_new_stage;
    ELSE
        SELECT * INTO v_new_stage FROM crm.pipeline_stages
        WHERE sort_order = v_old_stage.sort_order + 1 AND NOT is_closed_lost
        ORDER BY sort_order LIMIT 1;
    END IF;

    IF NOT FOUND THEN RAISE EXCEPTION 'No next stage found'; END IF;

    -- Calculate time in old stage
    SELECT NOW() - changed_at INTO v_time_spent
    FROM crm.deal_stage_history
    WHERE deal_id = p_deal_id
    ORDER BY changed_at DESC LIMIT 1;

    -- Update last history record with time spent
    UPDATE crm.deal_stage_history
```

```

SET time_in_stage = v_time_spent
WHERE history_id = (
    SELECT history_id FROM crm.deal_stage_history
    WHERE deal_id = p_deal_id ORDER BY changed_at DESC LIMIT 1
);

-- Insert new history record
INSERT INTO crm.deal_stage_history (deal_id, from_stage_id, to_stage_id,
changed_by)
VALUES (p_deal_id, v_deal.stage_id, v_new_stage.stage_id, p_user_id);

-- Update deal
UPDATE crm.deals
SET stage_id = v_new_stage.stage_id,
probability = v_new_stage.win_probability,
updated_at = NOW(),
closed_at = CASE WHEN v_new_stage.is_closed_won OR
v_new_stage.is_closed_lost
THEN NOW() ELSE NULL END
WHERE deal_id = p_deal_id;

RETURN format('Deal "%s" advanced to stage "%s"', v_deal.name,
v_new_stage.name);
END;
$$;

-----
-- FUNCTION 2: Get full contact 360 view
-----

CREATE OR REPLACE FUNCTION crm.get_contact_360(p_contact_id INTEGER)
RETURNS TABLE (
    contact_name TEXT,
    company TEXT,
    lifecycle_stage VARCHAR,
    open_deals BIGINT,
    total_deal_value NUMERIC,
    last_activity TIMESTAMPTZ,
    open_tasks BIGINT,
    campaigns_sent BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.first_name || ' ' || c.last_name,
        co.name,
        c.lifecycle_stage,
        COUNT(DISTINCT d.deal_id) FILTER (WHERE NOT ps.is_closed_won AND NOT
ps.is_closed_lost),
        COALESCE(SUM(d.amount) FILTER (WHERE NOT ps.is_closed_won AND NOT
ps.is_closed_lost), 0),
        MAX(a.occurred_at),

```

```

        COUNT(DISTINCT t.task_id) FILTER (WHERE t.status = 'Open'),
        COUNT(DISTINCT cc.id)
FROM   crm.contacts c
LEFT JOIN crm.companies co   ON c.company_id = co.company_id
LEFT JOIN crm.deals d        ON c.contact_id = d.contact_id
LEFT JOIN crm.pipeline_stages ps ON d.stage_id = ps.stage_id
LEFT JOIN crm.activities a    ON c.contact_id = a.contact_id
LEFT JOIN crm.tasks t        ON c.contact_id = t.contact_id
LEFT JOIN crm.campaign_contacts cc ON c.contact_id = cc.contact_id
WHERE  c.contact_id = p_contact_id
GROUP BY c.contact_id, c.first_name, c.last_name, co.name, c.lifecycle_stage;
END;
$$;

```

## 10. Triggers

```

-- TRIGGER 1: Auto-log activity when deal stage changes
CREATE OR REPLACE FUNCTION crm.trg_log_stage_change_activity()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_old_stage VARCHAR;
    v_new_stage VARCHAR;
BEGIN
    IF OLD.stage_id IS DISTINCT FROM NEW.stage_id THEN
        SELECT name INTO v_old_stage FROM crm.pipeline_stages WHERE stage_id =
OLD.stage_id;
        SELECT name INTO v_new_stage FROM crm.pipeline_stages WHERE stage_id =
NEW.stage_id;

        INSERT INTO crm.activities (type_id, user_id, deal_id, subject, occurred_at)
        SELECT 4, NEW.owner_id, NEW.deal_id,
            format('Stage changed: %s -> %s', v_old_stage, v_new_stage),
            NOW()
        WHERE NEW.owner_id IS NOT NULL;
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_deal_stage_activity
AFTER UPDATE OF stage_id ON crm.deals
FOR EACH ROW EXECUTE FUNCTION crm.trg_log_stage_change_activity();

-- TRIGGER 2: Soft delete cascade - mark contacts deleted when company deleted
CREATE OR REPLACE FUNCTION crm.trg_company_soft_delete_cascade()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.deleted_at IS NOT NULL AND OLD.deleted_at IS NULL THEN
        UPDATE crm.contacts SET deleted_at = NEW.deleted_at
        WHERE company_id = NEW.company_id AND deleted_at IS NULL;
    END IF;
    RETURN NEW;
END;
$$;

```



```

        END IF;
        RETURN NEW;
    END;
$$;

CREATE TRIGGER trg_company_cascade_delete
    AFTER UPDATE OF deleted_at ON crm.companies
    FOR EACH ROW EXECUTE FUNCTION crm.trg_company_soft_delete_cascade();

```

## 11. Performance Optimization

| Index                              | Table      | Reason                            |
|------------------------------------|------------|-----------------------------------|
| idx_activities_contact (composite) | activities | Contact timeline ordered by date  |
| idx_contacts_fts (GIN)             | contacts   | Full-text search on name + title  |
| idx_tasks_open (partial)           | tasks      | Only open tasks in rep dashboards |
| idx_deals_close                    | deals      | Pipeline by close date queries    |
| idx_companies_custom (GIN)         | companies  | JSONB custom field filtering      |

## 12. Extensions Used

```

CREATE EXTENSION IF NOT EXISTS pg_trgm; -- fuzzy contact search
CREATE EXTENSION IF NOT EXISTS btree_gist; -- for EXCLUDE constraints if needed

```

## 13. Testing Guide

```

-- TEST 1: Advance deal through stages
SELECT crm.advance_deal_stage(1, 2); -- Should advance to next stage
SELECT crm.advance_deal_stage(1, 2, 8); -- Close Won

-- TEST 2: Contact 360 view
SELECT * FROM crm.get_contact_360(2);

-- TEST 3: Pipeline report (Q1)
-- Verify weighted amounts are calculated correctly

-- TEST 4: Soft delete company
UPDATE crm.companies SET deleted_at = NOW() WHERE company_id = 3;
SELECT contact_id, deleted_at FROM crm.contacts WHERE company_id = 3;
-- All contacts should also be soft-deleted

-- TEST 5: Activity auto-logged on stage change (from trigger)
UPDATE crm.deals SET stage_id = 7 WHERE deal_id = 2;

```

```
SELECT subject FROM crm.activities WHERE deal_id = 2 ORDER BY occurred_at DESC LIMIT 1;
```

## 14. Extension Challenges

1. **Lead Scoring:** Add a `lead_score` column computed from activity count, deal amount, company size, and recency. Write a function that recalculates scores weekly using `pg_cron`.
2. **Territory Management:** Add geographic territories and assign reps by region. Write queries that show pipeline by territory and enforce territory-based deal ownership.
3. **Forecasting:** Build a `forecast_submissions` table where managers submit monthly revenue forecasts. Compare forecasts to actual closed deals with variance analysis.
4. **Duplicate Detection:** Write a function that finds potential duplicate contacts using trigram similarity on name + email + company name. Present duplicates for human review before merging.
5. **Webhook Outbox:** Add an `outbox_events` table. Whenever a deal is won or a contact is created, insert an event. Write a background process simulation that drains the outbox to an external webhook.

## 15. What You Learned

| Skill                       | Demonstrated By                                               |
|-----------------------------|---------------------------------------------------------------|
| SCD history table           | <code>deal_stage_history</code> records all stage transitions |
| Soft deletes                | <code>deleted_at</code> on contacts and companies             |
| JSONB custom fields         | Per-company/contact flexible attributes                       |
| Funnel analysis             | Lifecycle stage conversion rates                              |
| Self-referencing FK         | Users manager hierarchy, contacts reports-to                  |
| Temporal calculations       | Days to close, time in stage, SLA tracking                    |
| State machine trigger       | Stage change auto-logs activity                               |
| Cascade soft delete trigger | Company delete cascades to contacts                           |